

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 4

5 Programming in C and C++ (djc11+am21)

Bit-level
operations

- (a) 14-bit words are used to represent a certain set of natural numbers including zero. The least-significant five bits contain an unsigned binary-encoded mantissa value. The remaining nine bits represent an unsigned binary-encoded, bitwise left shift to be applied to the mantissa to obtain the represented value.
- (i) Give two functions, coded in C, that respectively convert an encoded value to its nearest 32-bit unsigned integer and to its nearest double-precision floating-point number. What problem(s) arise? [6 marks]

Answer:

```
unsigned int fourteen_to_int(int n14)
{
    unsigned int mant = n14 & 0x1F;          // Low five bits
    unsigned int ex = (n14 >> 5) & 0x1FF;    // High nine bits
    return mant << ex;
}
```

[Answered assuming 32-bit integers and double max of 10^{308} , but full credit for any reasonable assumptions.] A problem is that exponents above $32-5=27$ are likely to overflow, since the value is beyond the unsigned integer range. [There is a redundancy problem with the encoding, perhaps solved using a hidden bit approach, but that is not a problem arising in the requested functions.]

```
double fourteen_to_f64(int n14)
{
    unsigned int mant = n14 & 0x1F;          // Low five bits
    unsigned int ex = (n14 >> 5) & 0x1FF;    // High nine bits
    return (double)mant * exp(ex * log(2));
}
```

[Not a problem so not needed for full credit: The maximum value is 31×2^{511} which is (geometrically) only half way to f64's 10^{308} .]

- (ii) A packed array of such 14-bit words is stored in memory. Packed means no memory bits are unused, so the stored words may cross byte boundaries. Write a C function to implement the update operation for a 14-bit word held in the array. Its three arguments are an **unsigned char *** pointer to the base of the packed array, an integer index and an integer holding the 14-bit word to be stored. You may assume unaligned loads and stores of 32-bit words is supported. [6 marks]

Answer:

```
unsigned int lomask(int bits) { return (bits==32)? -1: (1<<bits)-1; }
unsigned int himask(int bits) { return ~lomask(bits); }
void update(unsigned char *b0, int offset, int nv)
{ unsigned int *bb = (unsigned int) b0;
  int boff = (offset * 14) / 32, brem = (offset * 14) % 32;
  bb[boff] = (bb[boff] & lomask(brem)) | (nv<<brem);
  int spill = 32 - 14 - brem;
```

```
    if (spill > 0)
        bb[boff+1] = (nv>>spill) & lomask(spill)) | (bb[boff+1] & himask(32-spill));
}
```

- (b) All calls to `malloc()` in a user program in C are to be replaced with calls to `my_malloc`.

- (i) Provide an implementation of `void *my_malloc(size_t)` that invokes the system's underlying `malloc` but which adds 16 bytes of padding at the start and end of each allocated region which is initialised with a distinctive bit pattern. [3 marks]

Answer:

```
void *my_malloc(int bytes)
{ assert(sizeof(int)==4);
  int *rr = malloc(bytes + 32);
  for (int i=0; i<4; i++) {
      rr[i] = 0x50501234 << i;
      rr[bytes/4+i] = 0xDeadBeef << i;
  }
  return (void *) rr + 4;
}
```

- (ii) Provide a companion `my_free` function that checks for any changes to the starting pattern, reporting appropriately, or else continues to invoke the system's `free`. [3 marks]

Answer:

```
void my_free(void *pp)
{ int *rr = (int *) (pp-16);
  for (int i=0; i<4; i++)
      if (rr[i] != 0x50501234 << i)
      {
          fprintf(stderr, "my_malloc corruption %p\n", pp);
          return;
      }
  free ((void *)rr);
}
```

Not required: To check the end pattern requires knowledge of the allocated length. For this exercise, this could be stored in the start pattern (eg make the start pattern the length plus a 96-bit CRC) or, if we are lucky, invoking a system-specific call such as `user_size` provided in many `malloc` implementations, although this may have rounded up the user's length. However, the general approach is to build the sentinel functionality into the user-space heap management system that invokes the `sbrk` or equivalent system call to get the raw heap space from the O/S.

- (iii) What might be the motivation for introducing `my_malloc`? [2 marks]

Answer: A common problem with C programs is accidental memory corruption arising from out-of-bounds array writes or incorrect assumptions of structure pointers (and

incorrect downcasts in C++). The sentinel pattern is likely to be disrupted, giving probabilistic detection of errors and small overruns will not corrupt a neighbouring heap object.
